

Opus Publica

RC2 Evolution Roadmap

From Human-Enforced to Self-Enforcing Governance

Version 1.0 (Draft for RC2 Planning)

Issued 09 August 2026

Document Type	In-Depth Research / Technical Documentation (Roadmap)
Issuing Authority	Office of the Chief Systems Architect, Opus Publica
Audience	Engineering Leadership, Architects, RC2 Planning Team, External Auditors
Classification	Strategic — RC2 Planning Authority
Builds On	Constitution Vol I (303pp) + Vol II (269pp) + Playbook (196pp) + Traceability Matrix (191pp) + Certification Checklist (326pp)
Effective Date	09 August 2026
Review Cadence	Monthly during RC2 design; quarterly after RC2 release
Document ID	OP-RC2-ROADMAP-001

RC1 is governed by humans reading documents. RC2 SHALL be governed by machines executing the same documents as code. This Roadmap specifies how.

Chapter 1 — The Constitutional Inflection Point

1.1 The Document at the Limit of Human Enforcement

Opus Publica's RC1 constitutional hierarchy is now complete: 1,285 pages across five documents, 346,000 words, 600+ certification criteria, 38 engineering invariants, 47 non-negotiable rules, 276 canonical fields, 34 events, 10 SLOs, 103 thresholds, and a 13-node traceability chain. By any measure, it is a comprehensive governing standard. It is also, as the post-RC1 review observed, a document at the limit of human enforcement. An engineer who must verify, by hand, that a single pull request satisfies the forty-seven non-negotiable rules is an engineer who will verify some of them, approximate others, and miss the rest. A Release Certification Authority who must, by hand, evaluate 600+ criteria per release is an authority who will sample, not exhaust, and the sampling will be biased toward the criteria the authority remembers. The RC1 Constitution is correct in what it specifies; it is insufficient in how it is enforced. RC2 is the release that closes that gap.

The thesis of this Roadmap is that governance at the scale Opus Publica now requires cannot remain a human discipline. The constitutional provisions are too numerous, the traceability chains too long, the certification criteria too fine-grained, and the evidence too voluminous for any human team to enforce continuously without error. The solution is not to reduce the Constitution — the provisions exist because the scholarly record requires them — but to encode the Constitution as a machine-readable, machine-executable, machine-enforced system. The five evolutionary improvements this Roadmap specifies — machine-readable constitution, automated traceability graph, continuous certification engine, governance dashboard, and self-auditing platform — are the mechanisms by which the Constitution ceases to be a document humans read and becomes a system machines execute. This is the constitutional inflection point: the moment Opus Publica's governance becomes self-enforcing rather than human-enforced.

The RC2 Prime Directive

Every constitutional provision that CAN be verified by a machine SHALL be verified by a machine. Every constitutional provision that CAN be enforced by a machine SHALL be enforced by a machine. Human review SHALL be reserved for judgement, not for verification of facts that a machine can check. A constitutional provision that remains human-enforced after RC2 SHALL be justified by a written argument that no machine verification is possible — not by the absence of automation effort.

1.2 Background: The RC1 Foundation

The RC1 constitutional hierarchy is the foundation on which RC2 is built. The hierarchy consists of five documents, each with a distinct role:

Document	Pages	Role	RC2 Status
----------	-------	------	------------

Constitution Volume I	303	Specifies what the platform is (25 chapters).	Foundation; preserved.
Constitution Volume II	269	Specifies invariants, contracts, events, thresholds (11 chapters).	Foundation; preserved.
Engineering Playbook	196	Operationalizes every rule (20 chapters).	Foundation; procedures automated.
Traceability Matrix	191	Maps every requirement to evidence (16 chapters).	Replaced by live graph.
Certification Checklist	326	Objective pass/fail criteria (16 chapters).	Encoded as executable criteria.
Total	1,285	Complete RC1 hierarchy.	RC2 builds on, does not replace.

The RC1 hierarchy is technology-neutral and implementation-independent by design. This was a deliberate constitutional choice: a Constitution that names a specific framework becomes obsolete when the framework is replaced. RC2 preserves this neutrality at the Constitution layer — the Constitution continues to specify contracts, not technologies — but introduces a new layer below the Constitution: the Machine-Readable Constitution (`constitution.yaml`), which is the executable encoding of the Constitution's provisions in a specific technology stack. The technology stack is named in this Roadmap and in the ADRs it triggers; the Constitution itself remains neutral.

1.3 The Five Evolutionary Improvements

The post-RC1 review identified five evolutionary improvements, each of which addresses a specific limitation of the RC1 human-enforced model. The improvements are sequential in dependency but parallel in development: each can be built independently, but each unlocks the value of the others.

#	Improvement	RC1 Limitation	RC2 Mechanism
1	Machine-Readable Constitution	Provisions are prose in PDFs; no canonical machine-readable source.	<code>constitution.yaml</code> — a versioned YAML/JSON Schema encoding of every provision, owned in Git, generating every document.
2	Automated Traceability Graph	Traceability is a hand-maintained matrix; broken chains detected quarterly.	Live property graph (Neo4j) built from code, contracts, and CI metadata; orphan detection by graph query.

3	Continuous Certification Engine	600+ criteria verified by hand per release; sampling bias; slow cycle.	OPA/Rego policies + GitHub Actions; every criterion runs as a CI job; OSCAL Assessment Results emitted continuously.
4	Governance Dashboard	Compliance status visible only in periodic PDF reports; no real-time view.	Grafana (operators) + Backstage Scorecards (developers); live constitutional health, SLO burn, certification coverage.
5	Self-Auditing Platform	Audits are periodic; orphan artifacts, stale evidence, drift detected late.	Nightly + continuous self-audit per NIST SP 800-137 ISCM; anomaly detection on audit log; drift detection on infra and constitution.

1.4 Scope and Non-Goals

This Roadmap specifies the architecture, technology selection, migration path, and governance framework for the RC2 evolution. It does not specify the RC2 product features (multi-tenancy, plugin SDK, AI assistance) except where they intersect with the governance evolution. RC2 product features are specified in Constitution Volume I Chapter 25 (Future Evolution) and will be detailed in separate RC2 product documents. This Roadmap's scope is the governance machinery that enables those product features to be built and certified under a self-enforcing regime.

- **In scope:** the governance technology stack (OPA, Neo4j, Backstage, Grafana, OSCAL).
- **In scope:** the migration path from RC1's human-enforced model to RC2's self-enforcing model.
- **In scope:** the constitutional amendments required to authorize the RC2 architecture.
- **Out of scope:** the RC2 product features (multi-tenancy, plugins, AI) — specified elsewhere.
- **Out of scope:** the replacement of the RC1 Constitution — RC2 builds on it, does not replace it.
- **Out of scope:** specific vendor selection beyond the canonical technology choices — those are ADRs.

1.5 Definitions

Term	Definition
Machine-Readable Constitution	A versioned YAML/JSON encoding of every constitutional provision, owned in Git, that is the single source of truth from which every human-readable

	document is generated.
Policy-as-Code	The discipline of expressing governance policies as versioned, testable, reviewable code, enforced at deploy/runtime. NIST SP 800-204C defines the term.
Continuous Compliance	Compliance verified continuously (per-commit, per-deploy, per-event) by automated tooling, rather than periodically by human auditors. ISACA frames this as “continuous assurance.”
Self-Enforcing Governance	A governance model in which the constitutional provisions are enforced by the platform itself, with humans reserved for judgement, not verification.
OSCAL	Open Security Controls Assessment Language — a NIST-led machine-readable format for security control documentation, including Assessment Results.
SLSA	Supply-chain Levels for Software Artifacts — an OpenSSF framework defining supply-chain integrity levels (L1-L4) and provenance attestations.
ISCM	Information Security Continuous Monitoring — NIST SP 800-137’s 6-phase lifecycle (Define, Establish, Implement, Analyze, Report, Respond, Review).

Chapter 2 — Pillar 1: Machine-Readable Constitution

2.1 Purpose

This chapter specifies the architecture of the Machine-Readable Constitution (MRC) — the single, versioned, machine-readable source of truth for every constitutional provision of Opus Publica. The MRC replaces the RC1 model in which provisions are authored as prose in Word documents and cross-referenced by humans. In RC2, provisions are authored as structured data in a Git repository, and every human-readable document (the Constitution, the Playbook, the Traceability Matrix, the Certification Checklist) is generated from the MRC. The MRC is the canonical source; the documents are derived views. This eliminates the consistency problem the post-RC1 review identified: maintaining thousands of cross-references across five documents is intractable for humans but trivial for a generator.

2.2 The Canonical Format

The MRC is encoded in YAML 1.2 (the current authoritative spec, YAML 1.2.2), with JSON Schema Draft 2020-12 as the validation vocabulary. YAML is chosen because it is human-readable (reviewable in pull requests), JSON-superset (consumable by every tool in the stack), and the dominant format for Kubernetes, GitHub Actions, OPA data files, and Backstage catalog-info. JSON Schema 2020-12 is chosen because it is the lingua franca of API and contract validation across OPA, Conftest, OpenAPI 3.1, and Backstage — adopting it means the same schema is consumed by the API gateway, the CI gate, and the developer portal without translation drift.

CUE (Configure, Unify, Execute), a Google-developed data-validation language, is evaluated as the authoring tool for the parts of the MRC where constraints are non-trivial (state-machine transitions, cross-field invariants, conditional validation). CUE's type system can express invariants that JSON Schema cannot (e.g., “if field X is set, field Y MUST be null”), and it can generate JSON Schema, OpenAPI, and Go types from a single source. The RC2 architecture uses CUE for authoring and JSON Schema for runtime validation; CUE is the source, JSON Schema is the compiled output. This is the same pattern a compiler uses: authors write in the high-level language; the runtime executes the compiled form.

2.3 The constitution.yaml Structure

The MRC is organized as a directory tree in Git, with each top-level file or subdirectory corresponding to a provision class. The structure is:

```
constitution/
  meta.yaml           # version, effective-date, supersedes, owner
  invariants/         # Vol II Ch 26: 38 invariants
    identity/         # INV-I-01..07
                      INV-I-01.yaml
                      INV-I-02.yaml
                      ...
```

immutability/	#	INV-M-01..05
resolvability/	#	INV-R-01..05
completeness/	#	INV-C-01..07
legality/	#	INV-L-01..06
auditability/	#	INV-A-01..04
conservation/	#	INV-V-01..04
rules/	# Vol I Ch 3: 47	non-negotiable rules
architecture/	#	ARCH-001..007
data/	#	DATA-001..007
publication/	#	PUB-001..008
external/	#	EXT-001..007
operations/	#	OPS-001..008
process/	#	PROC-001..010
contracts/	# Vol II Ch 27: 20	subsystem contracts
		SUB-01-submission.yaml
		SUB-02-review.yaml
		...
		SUB-20-storage.yaml
fields/	# Vol II Ch 28: 276	canonical fields
article/	#	45 fields
author/	#	28 fields
journal/	#	27 fields
		...
events/	# Vol II Ch 29: 34	events
		EVT-01-ArticleSubmitted.yaml
		...
slos/	# Vol II Ch 32: 10	SLOs
		SLO-01-availability.yaml
		...
thresholds/	# Vol II Ch 31: 103	thresholds
		latency/
		throughput/
		...
state-machine/	# Vol I Ch 11: publication	state machine
		states.yaml
		transitions.yaml
		illegal-transitions.yaml
certification/	# Vol I Ch 8 + Vol II Ch 22:	certification criteria
criteria/	# CC-*	criteria, one file per criterion
		pillars.yaml
gate.yaml	#	Production Acceptance Gate definition
schemas/	# JSON Schema 2020-12	definitions
		invariant.schema.json
		rule.schema.json
		contract.schema.json
		...
policies/	# OPA/Rego	policy modules (one per criterion)
		INV-I-01.rego

```

generators/
gen_oscal.py
PUB-001.rego
...
# Document generators
gen_constitution_vol1.py
gen_constitution_vol2.py
gen_playbook.py
gen_traceability_matrix.py
gen_certification_checklist.py
# OSCAL Assessment Results generator

```

2.4 The Provision Schema

Every provision (invariant, rule, contract field, event, SLO, threshold) conforms to a uniform schema. The schema is defined in JSON Schema 2020-12 and validated on every commit. The uniform schema is what permits a single generator to produce every human-readable document and a single policy engine to enforce every provision. The schema fields are:

Field	Type	Content
id	string	Stable identifier (e.g., INV-I-01, PUB-001).
class	enum	Provision class: invariant rule contract field event slo threshold state criterion.
statement	string	One-line statement of the provision.
source_chapter	string	Constitution chapter reference (e.g., "Vol II Ch 26 §26.1").
severity	enum	SEV-1 SEV-2 SEV-3.
owner	string	Owning team (e.g., "Data Engineering").
predicate	string	Formal predicate (for invariants) or condition (for rules).
verification_method	string	How the provision is verified (procedure).
required_evidence	object	Evidence type, format, max age, storage location.
verifier	enum	rca qa automated auditor.
automation	object	Policy module (Rego), CI job, monitor, alert.
runtime_monitor	string	Monitor ID and cadence (e.g., "INV-

		MON-I-01, hourly").
slo	string	Associated SLO ID (if applicable).
certification_rule	string	Associated CC-* criterion ID(s).
traceability	object	Architecture, ADR, DB, API, code, storage, test, evidence references.
version	string	SemVer of this provision (e.g., 1.0.0).
amended	array	Amendment history (date, sponsor, rationale).
status	enum	active deprecated retired.

2.5 Example: INV-I-02 as Machine-Readable Constitution

The following YAML is the machine-readable encoding of invariant INV-I-02 (Exactly one DOI per published article version). This single file is the source of truth for: the Constitution Vol II Ch 26 prose; the Certification Checklist CC-INV-I-02 criterion; the Traceability Matrix Ch 4 row; the OPA policy that enforces the invariant at runtime; the Prometheus alert that fires on violation; and the OSCAL Assessment Result that records the certification decision.

```
# constitution/invariants/identity/INV-I-02.yaml
id: INV-I-02
class: invariant
statement: >-
  Exactly one DOI per published article version.
source_chapter: "Vol II Ch 26 \u00a726.1"
severity: SEV-1
owner: Platform Engineering
predicate: >-
  \u2203! doi : PublishedVersion(v) \u2192 HasDOI(v, doi) \u2227 Unique(doi)
verification_method: >-
  SQL: COUNT(distinct dois WHERE article_version_id = V) = 1 for every published V;
  AND COUNT(article_versions WHERE doi = D) = 1 for every registered D.
  Crossref REST API reconciliation confirms DOI resolves and metadata matches.
required_evidence:
  type: reconciliation_report
  format: csv
  max_age: 1h
  storage: certification-repository/invariants/INV-I-02/
verifier: automated
automation:
  policy_module: policies/INV-I-02.rego
  ci_job: .github/workflows/verify-invariants.yml [INV-I-02]
  monitor: INV-MON-I-02 # hourly scan
  alert: INV-I-02-violation # SEV-1, pages on-call
```

```

runtime_monitor:
    id: INV-MON-I-02
    cadence: hourly
    implementation: reconciliation-job-python
slo: null # invariants are binary; not SLO-tracked
certification_rule: CC-INV-I-02
traceability:
    architecture: [ARCH-publication-service, ARCH-doi-minter]
    adr: [ADR-007-doi-minting-strategy]
    database: [dois, article_versions]
    api: [POST /api/v1/articles/{id}/mint-doi]
    code: [services/doi-minter/, services/publication/guard.py]
    storage: [] # no storage artifacts; DOI is a record
    test: [TEST-INV-I-02-uniqueness, TEST-INV-I-02-crossref-reconciliation]
    evidence: [EVID-2026-INV-I-02-latest]
version: 1.0.0
amended: []
status: active

```

2.6 Document Generation

Every human-readable document in the RC1 hierarchy is regenerated from the MRC on every commit. The generators are Python scripts (using python-docx for Word output, reportlab for PDF) that read the constitution/ directory, apply the document-specific template, and produce the .docx/.pdf artifact. The generation is a CI step: a PR that modifies a provision regenerates every affected document, and the regenerated documents are committed alongside the provision change. This guarantees that the human-readable documents are never out of sync with the machine-readable source.

Document	Generator	Input	Output
Constitution Vol I	gen_constitution_vol1.py	constitution/ {invariants,rules,contracts ,fields,events,slos,thresholds,state-machine}/	Vol_I_v{semver}.docx
Constitution Vol II	gen_constitution_vol2.py	constitution/ {invariants,contracts,fields,events,thresholds}/	Vol_II_v{semver}.docx
Engineering Playbook	gen_playbook.py	constitution/ + playbook-templates/	Playbook_v{semver}.docx
Traceability Matrix	gen_traceability_matrix.py	constitution/ + traceability edges (from Neo4j export)	Traceability_Matrix_v{semver}.docx
Certification Checklist	gen_certification_checklist.py	constitution/	Certification_Checklist_v{s

	t.py	certification/criteria/	emver}.docx
OSCAL Profile	gen_oscal.py	constitution/ (all provisions)	opus-publica-rc2-oscal-profile.json
OSCAL Assessment Results	gen_oscal_ar.py (runtime)	constitution/certification/ + CI run results	opus-publica-rc2-ar-{release-id}.json

2.7 Constitutional Diffing

Because the MRC is structured data in Git, constitutional evolution becomes a diffable, automatable process. When a provision is amended, the change is a pull request that modifies one or more YAML files. The CI pipeline, on every such PR, produces a constitutional diff that answers the questions the post-RC1 review posed:

- **Question 1:** which requirements changed (added, modified, removed, deprecated)?
- **Question 2:** which evidence artifacts are now stale (because the criterion changed)?
- **Question 3:** which APIs became invalid (because the contract changed)?
- **Question 4:** which tests require regeneration (because the verified provision changed)?

The diff is produced by normalizing both versions of the MRC with `yq` (mikefarah/yq), computing a structured diff, and generating a JSON Patch (RFC 6902) that represents the change as a sequence of add/remove/replace operations. The JSON Patch is itself a node in the traceability graph, recording the amendment as a first-class event. The diff classification uses Conventional Commits (feat = additive, fix = corrective, BREAKING CHANGE = MAJOR bump) and the Confluent Schema Registry compatibility modes (BACKWARD, FORWARD, FULL, NONE) to mechanically determine whether the change is breaking. A breaking change automatically routes the PR through the Production Acceptance Gate process rather than the routine-merge path.

2.8 The Single Source of Truth

The MRC Is the Constitution

In RC2, the Machine-Readable Constitution (the constitution/ Git repository) IS the Constitution. The human-readable documents (Word, PDF) are generated views, not authoritative sources. An engineer who finds a discrepancy between a Word document and the MRC SHALL treat the MRC as correct and file a bug against the generator. This inverts the RC1 relationship, in which the Word documents were authoritative and the machine-readable forms were aspirational. The inversion is the foundational shift of RC2.

Chapter 3 — Pillar 2: Automated Traceability Graph

3.1 Purpose

This chapter specifies the Automated Traceability Graph (ATG) — a live, machine-built property graph that replaces the RC1 Traceability Matrix's hand-maintained tables. The ATG is the operational form of Volume II Chapter 23 (Knowledge Graph Constitution): a directed graph whose nodes are engineering artifacts (requirements, architecture components, ADRs, database tables, API endpoints, code modules, storage objects, workflows, UI components, tests, runtime evidence, certification records, release records) and whose edges are traceability relationships (realizes, justified-by, backed-by, exposed-by, implemented-by, stored-in, exercised-by, surfaced-by, verified-by, produces, certified-by, released-in). The ATG is built automatically from the codebase, the MRC, and CI metadata; it is queried by graph traversal; and its integrity is verified continuously by graph queries that detect orphan nodes, broken edges, and stale evidence.

3.2 The Graph Data Model

The ATG is implemented on a property-graph database. Neo4j is the reference implementation (market-leading, Cypher query language, open-source community edition); Amazon Neptune and Memgraph are evaluated as alternatives in ADR-TBD. The property-graph model is chosen over a relational model because traceability queries are graph traversals (find all tests that verify requirements realized by components that depend on service X), and graph traversals are $O(\text{depth})$ in a graph database but $O(\text{joins}^{\text{depth}})$ in a relational database. The 13-node traceability chain established in RC1 (Figure 1.1) maps directly to 13 node types in the graph.

Each node carries properties drawn from the MRC (the provision's id, severity, owner, etc.) and from the artifact it represents (the code module's path, the test's ID, the evidence's hash and timestamp). Each edge carries a type (the traceability relationship), a source, a timestamp, and a confidence (high for machine-detected edges, medium for declared edges, low for inferred edges). The confidence property permits the self-auditing platform (Chapter 6) to flag low-confidence edges for human review.

3.3 Graph Construction

The ATG is constructed by three builders that run in CI on every commit and on every release:

Builder	Input	Nodes Produced	Edges Produced
Constitution Builder	constitution/ (MRC)	Requirement nodes (one per provision).	None (requirements are the graph's roots).
Code Builder	Source code + OpenAPI 3.1 specs + catalog-info.yaml	Architecture, ADR, Database, API, Code, Storage nodes.	implemented-by, exposed-by, backed-by, stored-in edges (from code annotations and

			OpenAPI).
CI Builder	CI run results + test reports + evidence artifacts	Test, Evidence, Certification, Release nodes.	verified-by, produces, certified-by, released-in edges (from CI metadata).

The Code Builder uses AST analysis and annotation parsing to detect traceability edges automatically. A code module annotated with `@requires(INV-I-02)` produces an implemented-by edge from the INV-I-02 requirement node to the code-module node. An OpenAPI 3.1 spec with `x-traces-to: PUB-001` produces an exposed-by edge. A database migration with `--owner INV-C-02` produces a backed-by edge. This automatic edge detection is what makes the ATG live rather than hand-maintained: the edges are byproducts of normal development, not separate documentation labor.

3.4 Graph Queries

The ATG is queried in Cypher. The following queries are the operational form of the RC1 Traceability Matrix's audit procedure (Ch 14), restated as graph traversals. Each query runs in milliseconds, replacing the RC1 manual audit that took hours.

```
// Query 1: Orphan requirements (no architecture realization)
MATCH (r:Requirement)
WHERE NOT (r)-[:realizes]->(:Architecture)
RETURN r.id, r.statement, r.severity

// Query 2: Orphan tests (verify no requirement)
MATCH (t:Test)
WHERE NOT (t)-[:verified_by]->(:Requirement)
RETURN t.id, t.target

// Query 3: Stale evidence (older than max_age for its class)
MATCH (e:Evidence)
WHERE e.timestamp < datetime() - duration({hours: e.max_age_hours})
RETURN e.id, e.type, e.timestamp, e.max_age_hours

// Query 4: Broken traceability chain (requirement with no test)
MATCH (r:Requirement {severity: 'SEV-1'})
WHERE NOT (r)-[:verified_by]->(:Test)
RETURN r.id, r.statement

// Query 5: Full chain traversal (requirement to release)
MATCH path = (r:Requirement {id: 'INV-I-02'})-[:realizes]->(a:Architecture)-[:implemented_by]->(c:Code)-[:stored_in]->(s:Storage),
(r)-[:verified_by]->(t:Test)-[:produces]->(e:Evidence)-[:certified_by]->(cert:Certification)-[:released_in]->(rel:Release)
RETURN path
```

```
//      Query      6:      Constitutional      references      to      deleted      modules
MATCH      (r:Requirement)-[:implemented_by]->(c:Code)
WHERE      NOT      c.exists_in_codebase      #      cross-check      with      live      code      scan
RETURN      r.id,      c.path
```

3.5 Graph Integrity Verification

The 8 integrity rules from RC1 (Traceability Matrix Ch 2 §2.5) are encoded as Cypher queries that run continuously. A rule violation produces a graph node of type IntegrityViolation, which triggers an alert and blocks the next release. The rules and their query encodings are:

Rule	Cypher Encoding (Summary)	Violation Response
IR-01: Every requirement traces to architecture	MATCH (r:Requirement) WHERE NOT (r)-[:realizes]->(:Architecture) RETURN r	Orphan requirement; blocks release; ticket to owner.
IR-02: Every architecture traces to an ADR	MATCH (a:Architecture) WHERE NOT (a)-[:justified_by]->(:ADR) RETURN a	Undocumented design; blocks release; ADR required.
IR-03: Every API traces to a test	MATCH (api:API) WHERE NOT (api)-[:tests]->(:Test) RETURN api	Unverified contract; blocks release; test required.
IR-04: Every test traces to evidence in last release	MATCH (t:Test) WHERE NOT (t)-[:produces]->(:Evidence {release: lastRelease}) RETURN t	Test did not run; blocks release; investigate CI.
IR-05: SEV-1 requirements have complete chains	MATCH (r:Requirement {severity:'SEV-1'}) WHERE chain_has_gap(r) RETURN r	Critical defect; blocks release; CSA notified.
IR-06: Evidence within max age	MATCH (e:Evidence) WHERE e.timestamp < now() - e.max_age RETURN e	Stale evidence; treat as missing; refresh.
IR-07: No orphan nodes	MATCH (n) WHERE degree(n) = 0 RETURN n	Orphan artifact; ticket to owner; remediate or remove.
IR-08: Graph reconciled with Constitution on amendment	On MRC commit, run full reconciliation; flag discrepancies.	Drift detected; block release until reconciled.

3.6 Distributed Tracing Integration

The ATG integrates with OpenTelemetry distributed tracing to provide runtime traceability. Every Opus Publica service is instrumented with the OpenTelemetry SDK, and every event (EVT-01 through EVT-34) carries a trace context that flows through the outbox, the message queue, the consumer, and the external API call. The trace ID is recorded as a property on the Evidence node in the ATG, so an auditor can traverse from a requirement to the specific runtime trace that demonstrates its satisfaction. This

replaces RC1's manual evidence sampling with continuous, trace-level evidence: every publication event produces a trace, every trace is a node in the graph, and every trace is queryable by the audit procedure.

The integration uses the OpenTelemetry Collector to export traces to a backend (Tempo or Jaeger) and to the ATG's evidence-ingestion pipeline. The pipeline extracts the trace's spans, identifies the events each span represents, and creates Evidence nodes linked to the corresponding Test and Requirement nodes. The result is that the ATG's Evidence layer is continuously populated by real runtime traces, not by periodic manual evidence collection.

3.7 The Graph as Audit Substrate

The ATG is the audit substrate for RC2. An external auditor, given read access to the graph, can verify any requirement's satisfaction by a single Cypher query (Query 5 above). The auditor does not need to read 1,285 pages of documents; the auditor queries the graph. The auditor does not need to sample; the graph is exhaustive. The auditor does not need to trust the platform's self-attestation; the graph's edges are machine-detected from code and CI metadata, and the integrity rules continuously verify the graph's own consistency. This is what it means for a platform to be auditable by design: the audit is a query, not an expedition.

Chapter 4 — Pillar 3: Continuous Certification Engine

4.1 Purpose

This chapter specifies the Continuous Certification Engine (CCE) — the system that executes the 600+ certification criteria from the RC1 Certification Checklist automatically, in CI/CD, on every commit, every deploy, and every release. The CCE replaces the RC1 model in which the Release Certification Authority (RCA) evaluates criteria by hand per release. In RC2, the RCA's role shifts from evaluator to exception-handler: the CCE evaluates every criterion automatically, and the RCA intervenes only when a criterion FAILS and the implementing team disputes the failure or requests an exception. The CCE is the operational form of the post-RC1 review's directive: "GitHub Actions should verify it."

4.2 The Policy-as-Code Stack

The CCE is built on the policy-as-code stack: Open Policy Agent (OPA) as the policy engine, Rego as the policy language, Conftest as the pre-merge YAML/JSON policy runner, and GitHub Actions as the execution substrate. OPA is a CNCF Graduated project (graduated February 2021, with completed third-party security audits), satisfying the Constitution's technology-neutrality and audited-open-source bars. The stack is:

Layer	Technology	Role	Authority
Policy Engine	Open Policy Agent (OPA)	Evaluates Rego policies against input data; returns allow/deny decisions.	openpolicyagent.org
Policy Language	Rego	Declarative, Datalog-like language for expressing constitutional provisions as executable rules.	openpolicyagent.org/docs/policy-language
Pre-Merge Runner	Conftest	Runs Rego policies against YAML/JSON in CI; fails the build on violation.	conftest.dev
Execution Substrate	GitHub Actions	Runs Conftest + OPA on every PR; schedules nightly criterion runs; emits evidence.	docs.github.com/en/actions
Admission Controller	OPA-Kubernetes (Gatekeeper)	Enforces policies at deploy time; refuses non-compliant deployments.	openpolicyagent.org/docs/kubernetes

4.3 Criterion-to-Policy Mapping

Every criterion in the RC1 Certification Checklist (CC-INV-*, CC-ARCH-*, CC-PUB-*, etc.) is encoded as a Rego policy module in the MRC's policies/ directory. The mapping is one-to-one: one criterion, one policy, one CI job. The policy takes the MRC provision and the runtime evidence as input, evaluates the PASS condition, and returns a decision. The CI job runs the policy, records the decision, and uploads the evidence to the certification repository.

```
# policies/INV-I-02.rego
package opus_publica.invariants

import data.opus_publica.constitution.invariants.INV_I_02

# PASS: COUNT(distinct dois WHERE article_version_id = V) = 1
# for every published V
deny[msg] {
    some v
    count([d | d := input.dois[_]; d.article_version_id == v.id]) != 1
    msg := sprintf(
        "INV-I-02 FAIL: article version %s has %d DOIs (expected 1)",
        [v.id, count([d | d := input.dois[_]; d.article_version_id == v.id])])
}

# PASS: every registered DOI resolves via Crossref REST API
deny[msg] {
    some d
    d := input.dois[_]
    not input.crossref_reconciliation[d.value].resolved
    msg := sprintf(
        "INV-I-02 FAIL: DOI %s does not resolve via Crossref",
        [d.value])
}
```

4.4 The CI Workflow

The GitHub Actions workflow that runs the CCE has three trigger types, each with a different cadence and scope:

Trigger	Cadence	Scope	Gating
Pull Request	On every PR to main	Criteria whose provisions the PR touches (determined by ATG	Blocks merge on FAIL.

		query).	
Nightly	Cron: 0 2 * * *	All 600+ criteria.	Blocks next release on FAIL; opens tickets.
Release	On release-branch merge	All GATE criteria + Production Acceptance Gate (Ch 14).	Blocks release on FAIL; RCA exception only.

Each criterion's CI job produces a structured result: the criterion ID, the PASS/FAIL/N-A decision, the evidence artifact (uploaded to the certification repository), the verifier (automated), the timestamp, and the trace ID (from OpenTelemetry, if applicable). The result is emitted as an OSCAL Assessment Result (OSCAL-AR) JSON fragment, which the OSCAL collector assembles into the release's complete certification record. The OSCAL-AR format is the bridge to external auditors: any auditor with OSCAL tooling can consume the certification record without reading Opus Publica-specific formats.

4.5 The Production Acceptance Gate as Code

The RC1 Production Acceptance Gate (Certification Checklist Chapter 14) is a 26-criterion final gate that, in RC1, is a human ceremony. In RC2, the Gate is a GitHub Environment protection rule that requires every Gate criterion's CI job to PASS before the environment's deployment is permitted. The Gate is enforced by GitHub's Required Checks feature: a deployment to production cannot be initiated until every required check is green. The RCA's sign-off, in RC2, is the act of approving the deployment in GitHub (a button click), not the act of evaluating 26 criteria by hand. The criteria have already been evaluated by the CCE; the RCA is confirming that the automated evaluation is acceptable and that no exception is warranted.

The Gate as Code

The Production Acceptance Gate in RC2 is a GitHub Environment with Required Checks. Every criterion CC-GATE-* is a required check. A deployment to production is technically impossible until every check PASSES. There is no override below the Chief Systems Architect, and the CSA SHALL NOT override a Runtime Certification failure. The Gate is enforced by the platform, not by the RCA's discipline.

4.6 Supply-Chain Integrity

The CCE integrates supply-chain integrity controls that RC1 names as targets but does not operationalize. The integration uses three OpenSSF technologies: SLSA (Supply-chain Levels for Software Artifacts) for provenance, Sigstore (cosign, rekor, fulcio) for signing, and in-toto for attestations. Every Opus Publica release artifact (article PDF, JATS XML package, Docker image, Helm chart) is built by a trusted builder (GitHub Actions), signed with Cosign at build time, recorded in the Rekor transparency log, and verified

at deploy time by an OPA policy that refuses unsigned or improperly-signed artifacts. The SLSA L3 target named in Volume II Chapter 16 is achieved by this integration.

Control	Technology	RC2 Achievement
Provenance	SLSA L3	Every artifact carries a SLSA L3 provenance attestation linking it to its source commit, builder, and build steps.
Signing	Sigstore (Cosign)	Every artifact is signed with an ephemeral key derived from the GitHub Actions OIDC identity; no long-lived signing keys.
Transparency	Sigstore (Rekor)	Every signature is recorded in the Rekor append-only transparency log; auditable by anyone.
Attestation	in-toto Attestation Framework	Every certification criterion emits an in-toto Statement/Predicate attestation (SLSA Provenance, SPDX, vulnerability scan, test result); the certification record is a collection of attestations.
SBOM	CycloneDX 1.6 (ECMA-424) + SPDX 3.0 (ISO/IEC 5962:2021)	Every deployment artifact ships both SBOM formats, signed and recorded; queryable by the ATG and auditable by external parties.

4.7 The OSCAL Bridge

The CCE emits its results in OSCAL (Open Security Controls Assessment Language), a NIST-led machine-readable format for security control documentation. OSCAL defines a layered model: Catalog (controls), Profile (baseline selection), Component Definition (implementation), System Security Plan (SSP), Assessment Plan, Assessment Results (AR), and Plan of Action and Milestones (POA&M). The CCE uses the Assessment Results layer: every criterion run produces an OSCAL-AR JSON fragment, and the release's complete certification record is an OSCAL-AR document that an external auditor can consume with off-the-shelf OSCAL tooling. This is the bridge that makes Opus Publica's certification legible to FedRAMP, funder-compliance officers, and other auditors who operate in the OSCAL ecosystem.

Chapter 5 — Pillar 4: Governance Dashboard

5.1 Purpose

This chapter specifies the Governance Dashboard — the real-time visibility layer that replaces RC1's periodic PDF status reports. The Dashboard is the constitutional health of Opus Publica, visible at a glance, continuously updated, and accessible to every role: engineers, engineering leadership, the RCA, the Chief Systems Architect, and (with appropriate access) external auditors. The Dashboard is the post-RC1 review's fourth evolutionary improvement: “at that point, the Constitution becomes a living operational system rather than a collection of PDFs.”

5.2 The Two-Surface Architecture

The Dashboard is implemented on two complementary surfaces, each serving a different audience with different needs. Grafana serves operators (on-call engineers, SREs, the RCA) with real-time metrics, logs, traces, and alerts. Backstage serves developers (implementing engineers, tech leads) with the software catalog, scorecards, docs, and the scaffolder. The two surfaces are complementary: Grafana answers “is the platform healthy right now?”; Backstage answers “is my service compliant and mature?”. Both surfaces consume the same underlying data (Prometheus metrics, OSCAL-AR feed, ATG queries) via shared data sources, so the two views are always consistent.

Surface	Technology	Audience	Primary View
Operations Dashboard	Grafana (OSS) + Prometheus + Loki + Tempo	On-call, SRE, RCA, CSA.	Real-time constitutional health: invariant status, SLO burn, alert volume, trace coverage.
Developer Portal	Backstage (Spotify open-source) + Scorecards plugin	Engineers, tech leads, engineering managers.	Per-service compliance: scorecard grade, open findings, in-flight PRs, contract status.

5.3 The Constitutional Health Dashboard (Grafana)

The Constitutional Health Dashboard is a Grafana dashboard with the following panels, each backed by a Prometheus query or an OSCAL-AR data source. The dashboard is the single pane of glass for constitutional health; it is the first thing the RCA and the CSA consult each morning and the first thing on-call consults during an incident.

Panel	Query Source	Visualization	Alert Threshold
Invariant Status (38)	OSCAL-AR feed (per-invariant)	Stat panel: green/red count;	Any SEV-1 invariant FAIL →

invariants)	PASS/FAIL)	table of failing invariants.	page on-call.
Traceability Completeness	ATG query (orphan count)	Gauge: % of requirements with complete chains.	< 100% for SEV-1 requirements → ticket.
Certification Coverage	OSCAL-AR feed (criterion PASS count / total)	Stat: % criteria PASS in last 24h.	< 100% GATE criteria → block release.
Failed Criteria (24h)	OSCAL-AR feed	Table: criterion ID, FAIL reason, owner, age.	Any GATE FAIL > 24h → escalate.
Stale Evidence	ATG query (evidence age > max)	Table: evidence ID, type, age, owner.	Any SEV-1 evidence stale > 1h → ticket.
Broken Traceability Edges	ATG query (integrity rule violations)	Stat: count of IR-01..08 violations.	Any violation → ticket; SEV-1 violation → page.
SLO Status (10 SLOs)	Prometheus (SLI values, error budgets)	Multi-panel: per-SLO gauge + error budget burn rate.	Error budget exhausted → release freeze.
Release Readiness	OSCAL-AR feed + GitHub Environment checks	Traffic light: green (all Gate criteria PASS) / red (any FAIL).	Red → deployment blocked; RCA notified.
Audit Log Integrity	ATG query (chain hash verification)	Stat: chain verification PASS/FAIL.	Any FAIL → SEV-1 incident; CSA notified.
Supply-Chain Provenance	Rekor query (signed artifact count)	Stat: % artifacts with valid SLSA L3 provenance.	< 100% → deployment blocked.

5.4 The Developer Scorecard (Backstage)

The Developer Scorecard is a Backstage Scorecard that grades each of the 20 subsystems on the five certification pillars. The scorecard uses CQL (Check Quality Language) rules that query the OSCAL-AR feed and the ATG; each rule returns PASS/FAIL for one pillar on one subsystem. A developer opening their subsystem's Backstage page sees a red/yellow/green grade per pillar, the specific failing criteria, and a link to the failing criterion's MRC file. This closes the shift-left loop: the developer sees the constitutional impact of their work in the same portal where they are working, not in a separate audit report.

Pillar	Scorecard Rule (CQL)	Grade
Engineering Constitution	All CC-* criteria for this subsystem's provisions PASS in last 24h.	Green (all PASS) / Red (any FAIL)
Engineering Playbook	All Playbook procedures for this subsystem executed (CI evidence).	Green / Red
Runtime Certification	Five pillars PASS for this subsystem's last release.	Green / Yellow (4/5) / Red (< 4/5)
Production Acceptance Criteria	Operational readiness: on-call assigned, runbook updated,	Green / Red

	rollback tested.	
Traceability	No orphan nodes for this subsystem; all chains complete.	Green / Red

5.5 The Dashboard as Living Constitution

The Dashboard is the Constitution made visible. In RC1, the Constitution is a 1,285-page document that an engineer consults by search. In RC2, the Constitution is a Dashboard that an engineer consults by glance. The engineer does not need to know that INV-I-02 requires exactly one DOI per article version; the engineer needs to know that the invariant is green, and if it is red, the Dashboard tells them which article, which version, and which DOI is the problem. This is the shift from specification to operation: the Constitution is no longer a document to be read; it is a system to be observed. The Dashboard is the observation surface.

Chapter 6 — Pillar 5: Self-Auditing Platform

6.1 Purpose

This chapter specifies the Self-Auditing Platform (SAP) — the system that audits Opus Publica continuously, detecting orphan artifacts, broken traceability, stale evidence, governance drift, and constitutional anomalies without human initiation. The SAP replaces RC1's periodic audit model (quarterly integrity checks, annual external audit) with continuous assurance: the platform audits itself every night and every commit, and escalates only the findings that require human judgement. The SAP is the operational form of the post-RC1 review's fifth improvement: “that would turn governance into continuous assurance rather than periodic review.”

6.2 The ISCM Framework

The SAP is architected against NIST SP 800-137, “Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations.” SP 800-137 defines a 6-phase ISCM lifecycle that the SAP implements directly:

Phase	SP 800-137 Definition	Opus Publica RC2 Implementation
1. Define	Define the ISCM strategy based on risk tolerance, assets, controls.	The MRC defines the controls (38 invariants, 47 rules, 600+ criteria); risk tolerance is encoded in severity (SEV-1/2/3).
2. Establish	Establish metrics, sensors, frequency, and data sources.	Prometheus metrics, OSCAL-AR feed, ATG queries, OpenTelemetry traces; frequency per evidence class (1h for invariants, 24h for reconciliations).
3. Implement	Implement the ISCM program (tools, automation, workflows).	GitHub Actions (nightly + per-PR), OPA policies, Conftest, driftctl, anomaly detection on audit log.
4. Analyze	Analyze the data; identify anomalies, trends, risks.	Rule-based anomaly detection (known-bad patterns) + ML-based (Isolation Forest / Local Outlier Factor) for unknown-unknowns.
5. Report	Report findings to stakeholders.	Grafana dashboards (operators), Backstage scorecards (developers), OSCAL-AR (auditors), weekly summary to CSA.
6. Respond & Review	Respond to findings; review and	Auto-response (block release, open

	refine the ISCM program.	ticket) for GATE findings; human response for judgement findings; quarterly ISCM review.
--	--------------------------	--

6.3 The Nightly Self-Audit

The nightly self-audit is a GitHub Actions workflow (cron: 0 2 * * *) that runs every self-audit check and produces a nightly audit report. The report answers the questions the post-RC1 review posed, each as a graph query or a CI check:

Audit Question	Check	Source
Any orphan requirements?	ATG query IR-01: requirements with no architecture realization.	Graph query.
Any orphan APIs?	ATG query: API nodes with no implementing code module.	Graph query + code scan.
Any missing runtime evidence?	ATG query IR-04: tests with no evidence in last release.	Graph query.
Any stale evidence?	ATG query IR-06: evidence older than max age.	Graph query.
Any broken traceability edges?	ATG query IR-07: orphan nodes (degree 0).	Graph query.
Any certification gaps?	OSCAL-AR query: GATE criteria not PASS in last 24h.	OSCAL feed.
Any constitutional references to deleted modules?	ATG query: requirement nodes pointing to code paths that no longer exist.	Graph query + code scan.
Any infrastructure drift?	driftctl scan: live cloud state vs. Terraform state.	driftctl.
Any constitutional drift?	Rego policy: live /contracts/SUB-NN.yaml vs. version pinned in MRC.	Conftest.
Any audit-log anomalies?	Rule-based + ML (Isolation Forest) on audit log.	Anomaly detection.
Any SBOM vulnerabilities?	CycloneDX + vulnerability DB query.	Dependency track.
Any expired credentials?	Secret manager query: credentials past rotation deadline.	Secret manager API.

The nightly report is committed to the certification repository, summarized on the Grafana dashboard, and (for any SEV-1 finding) paged to the on-call engineer. The report is also the input to the weekly CSA review: the CSA consults the nightly reports from the past week, identifies trends, and either accepts the findings' remediation plans or escalates. The nightly audit is the floor of continuous assurance; the per-PR and per-event checks are the ceiling.

6.4 Continuous Anomaly Detection

The SAP layers two anomaly-detection approaches on the audit log. The rule-based layer detects known-bad patterns: two publication events for the same DOI with different content hashes; a state transition that bypasses the state machine; an audit record missing a required field; a credential used from an unusual geography. Rules are encoded as Rego policies and evaluated in real time as audit records are written. A rule match is a SEV-1 or SEV-2 incident, paged immediately. The rule layer is the primary auditor-trustable signal: deterministic, auditable, and explainable.

The ML-based layer detects unknown-unknowns: patterns that no rule anticipated but that deviate from the platform's normal behaviour. The layer uses unsupervised models (Isolation Forest and Local Outlier Factor are the reference algorithms, per the research literature) trained on the audit log's historical patterns. An ML anomaly does not trigger an automatic page; it triggers a ticket for human review. The ML layer is the early-warning radar: it surfaces events for human attention before they become incidents. The two layers are complementary: rules catch the known; ML catches the novel; together they provide defense-in-depth for the audit log.

6.5 Drift Detection

The SAP detects two kinds of drift. Infrastructure drift — the live cloud state diverging from the Terraform-declared state — is detected by driftctl (now maintained by Snyk), which runs nightly and reports any difference between Terraform state and actual cloud state. SEV-1/2 drift blocks the next release; SEV-3 drift is ticketed. Constitutional drift — a live /contracts/SUB-NN.yaml file diverging from the version pinned in the MRC — is detected by a Rego policy that compares the live contract against the MRC's reference. Constitutional drift is always a defect (the MRC is the source of truth) and always blocks the next release. The two drift detectors share a common pattern: declared state vs. actual state, with drift as a defect, not as a tolerated divergence.

6.6 The Self-Audit as Continuous Assurance

The SAP is the operational form of “continuous assurance,” a term formalized by ISACA (the professional association for IT audit, governance, and risk). Continuous assurance, in ISACA's framing, is “a paradigm move since compliance is consolidated as a constant state of operation instead of an annual milestone.” The SAP realizes this paradigm for Opus Publica: compliance is not a quarterly exercise but a continuous property of the running system, verified every night and every commit, with anomalies escalated to humans only when judgement is required. The external auditor's role, in RC2, shifts from periodic

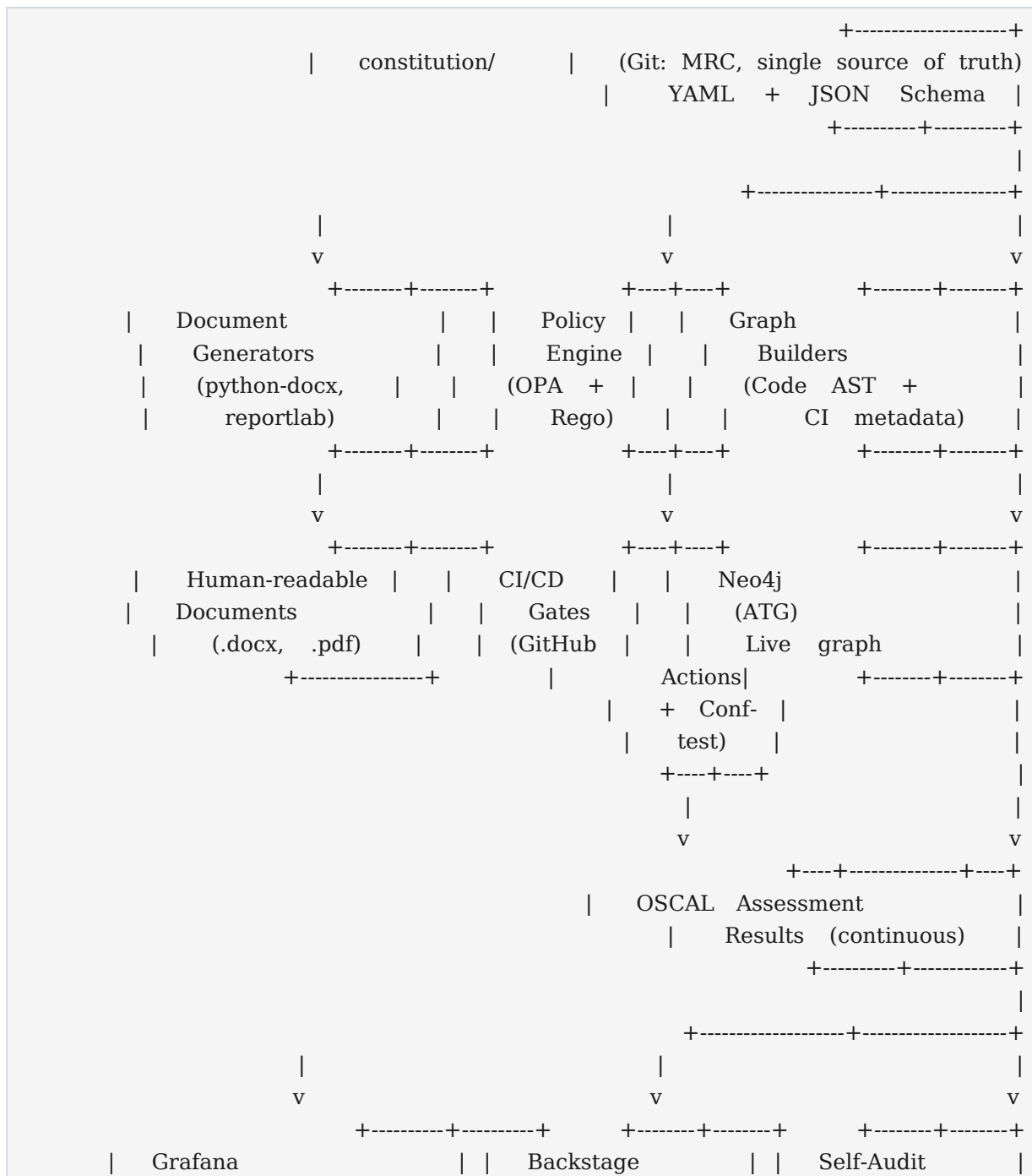
verification to periodic confirmation that the SAP itself is functioning — a far lighter-touch audit that consumes fewer external-auditor hours and produces higher confidence.

Chapter 7 — Implementation Architecture

7.1 Purpose

This chapter specifies the integrated architecture of the five RC2 pillars, showing how they compose into a single self-enforcing governance system. The architecture is the blueprint RC2 implementation will follow; it identifies the components, their interfaces, their data flows, and their deployment topology.

7.2 The Integrated Architecture



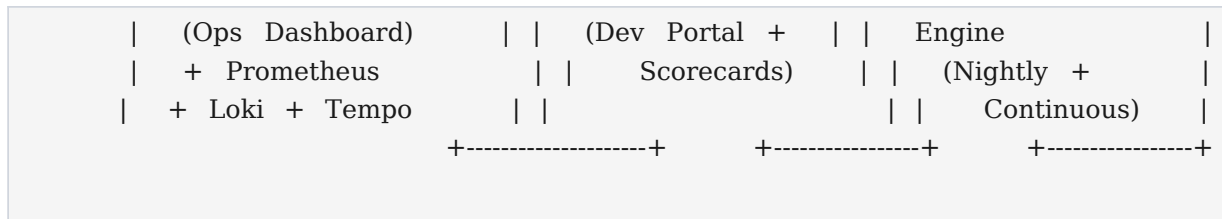


Figure 7.1: The RC2 Self-Enforcing Governance Architecture

7.3 Component Catalogue

Component	Technology	Role	Deployed As
MRC Repository	Git (GitHub)	Single source of truth for all provisions.	GitHub repo (opus-publica/constitution).
Document Generators	Python (python-docx, reportlab)	Generate human-readable docs from MRC.	CI job on every MRC commit.
Policy Engine	Open Policy Agent (OPA) + Rego	Evaluate provisions as executable policies.	Sidecar in each service + CI.
Pre-Merge Runner	Conftest	Run Rego on YAML/JSON in CI.	GitHub Actions step on every PR.
CI/CD Substrate	GitHub Actions	Execute criteria; emit evidence.	GitHub-hosted + self-hosted runners.
Admission Controller	OPA Gatekeeper	Enforce policies at deploy.	Kubernetes admission webhook.
Deploy Controller	Flux (or Argo CD)	GitOps reconcile live state to declared state.	Kubernetes controller.
Traceability Graph	Neo4j (or Amazon Neptune)	Live graph of artifacts and edges.	Managed graph DB.
Graph Builders	Python (AST + annotation parsers)	Build graph nodes/edges from code + CI.	CI job on every commit.
Distributed Tracing	OpenTelemetry SDK + Collector + Tempo (or Jaeger)	Runtime traces; evidence for ATG.	Sidecar + OTel Collector.
Metrics	Prometheus + OpenTelemetry metrics	SLI/SLO/error-budget metrics.	Kubernetes + Prometheus operator.
Logs	Loki (or Elasticsearch)	Structured logs for audit and debugging.	Kubernetes + Loki stack.
Operations Dashboard	Grafana (OSS)	Constitutional health dashboard for operators.	Kubernetes + Grafana.
Developer Portal	Backstage (Spotify OSS)	Catalog, scorecards, docs, scaffolder.	Kubernetes + Backstage.

Supply-Chain Signing	Sigstore (Cosign + Rekor + Fulcio)	Sign artifacts; transparency log.	CI step + Rekor (hosted).
SBOM Generation	CycloneDX 1.6 + SPDX 3.0	Generate SBOMs for every artifact.	CI step (syft + cyclonedx).
Provenance	SLSA L3 + in-toto attestations	Provenance attestations for every artifact.	CI step (slsa-github-generator).
Certification Format	OSCAL (NIST)	Machine-readable certification records.	CI step (oscal-cli).
Infrastructure Drift	driftctl (Snyk)	Detect cloud/TF drift.	Nightly CI job.
Anomaly Detection	Python (scikit-learn) + Rego rules	Detect audit-log anomalies (rule + ML).	Nightly + real-time.

7.4 Data Flow: A Pull Request Through the System

To make the architecture concrete, consider a pull request that modifies the DOI minter service. The PR's journey through the RC2 system is:

- Step 1.** The engineer opens a PR modifying `services/doi-minter/` and, incidentally, the contract `/contracts/SUB-10-doi-minter.yaml`.
- Step 2.** GitHub Actions triggers the pre-merge workflow. Conftest runs every Rego policy whose provisions the PR touches (determined by an ATG query: which provisions trace to the modified code?).
- Step 3.** The Code Builder runs, updating the ATG with new/modified nodes and edges. The Integrity Verifier runs the 8 integrity rules; any violation fails the PR.
- Step 4.** The Document Generators run, regenerating any human-readable document affected by the MRC change (if the contract YAML changed, the Traceability Matrix and Certification Checklist are regenerated).
- Step 5.** The CI runs the service's test suite, produces evidence (test logs, coverage, contract-test results), and uploads to the certification repository. The evidence is linked to the ATG as Evidence nodes.
- Step 6.** OSCAL-AR fragments are emitted for every criterion that ran; the fragments are assembled into the PR's certification preview.
- Step 7.** The Backstage Scorecard for the DOI Minter service updates to reflect the PR's impact (e.g., a pillar goes yellow if a criterion is now at-risk).
- Step 8.** The reviewer reviews the PR in GitHub, seeing the Conftest results, the ATG integrity check, the regenerated documents, and the Scorecard impact. The reviewer approves (or requests changes).
- Step 9.** On merge, Flux detects the change and reconciles the staging environment. The Gatekeeper admission controller enforces the deploy-time OPA policies.

10. **Step 10.** The release certification runs (per-release trigger); the Production Acceptance Gate evaluates all GATE criteria. If all PASS, the RCA approves the production deployment in GitHub.
11. **Step 11.** Flux deploys to production. OpenTelemetry traces flow; the ATG records the release; the Grafana dashboard updates; the nightly self-audit will verify the release's integrity at 02:00.

Chapter 8 — Migration Path from RC1 to RC2

8.1 Purpose

This chapter specifies the migration path from RC1's human-enforced governance to RC2's self-enforcing governance. The migration is not a flag-flip; it is a phased transition in which each pillar is built, validated, and activated before the next begins, and in which the RC1 human-enforced regime remains in force as a safety net until the RC2 machinery has demonstrated reliability. The migration is designed to introduce no governance gap: at every point in the transition, the platform's provisions are enforced by at least one regime (RC1 human, RC2 machine, or both).

8.2 The Four Phases

Phase	Duration	Goal	Exit Criteria
Phase 0 — Foundation	4 weeks	Stand up the MRC repository; encode the 38 invariants and 47 rules as YAML; validate against JSON Schema.	MRC repo live; 85 provisions encoded; JSON Schema validation passes in CI.
Phase 1 — Policy Encoding	6 weeks	Encode the 85 provisions as Rego policies; run Conftest in non-blocking mode (report-only) alongside RC1 human review.	85 Rego policies written; Conftest runs on every PR (report-only); parity with human review > 95%.
Phase 2 — Graph and Dashboard	6 weeks	Stand up Neo4j; build the ATG from code and CI; deploy Grafana and Backstage; wire to OSCAL-AR feed.	ATG live with all 13 node types; Grafana dashboard live; Backstage scorecards live; nightly self-audit running.
Phase 3 — Enforcement Cutover	4 weeks	Switch Conftest from report-only to blocking; activate the Production Acceptance Gate as GitHub Required Checks; retire RC1 human review for automated criteria.	All GATE criteria enforced by CI; RCA sign-off is a GitHub button click; RC1 human review retired for automated criteria.

8.3 Phase 0: Foundation (Weeks 1-4)

Phase 0 stands up the Machine-Readable Constitution repository and encodes the highest-value provisions first: the 38 invariants (Vol II Ch 26) and the 47 non-negotiable rules (Vol I Ch 3). These 85 provisions are the backbone of the certification regime, and encoding them first means the subsequent phases have a concrete target. The phase also establishes the JSON Schema validation: every YAML file in the MRC is validated against its schema on every commit, so structural errors are caught immediately. The output of Phase 0 is a Git repository in which 85 provisions are machine-readable, schema-validated, and ready to be encoded as policy. The human-readable RC1 documents remain authoritative during this phase; the MRC is a parallel artifact, not yet the source of truth.

8.4 Phase 1: Policy Encoding (Weeks 5-10)

Phase 1 encodes the 85 provisions as Rego policies and runs Conftest in non-blocking (report-only) mode alongside the existing RC1 human review. The goal of this phase is to measure parity: does the automated evaluation agree with the human evaluation? Discrepancies are investigated: if the machine PASSES a criterion the human FAILED, the policy may be too lenient; if the machine FAILS a criterion the human PASSED, the policy may be too strict or the human may have missed something. Parity is measured as the agreement rate between machine and human across all PRs in the phase. The phase exits when parity exceeds 95% for two consecutive weeks, demonstrating that the Rego policies faithfully encode the constitutional intent. Throughout Phase 1, the RC1 human review remains the binding regime; the machine evaluation is advisory.

8.5 Phase 2: Graph and Dashboard (Weeks 11-16)

Phase 2 stands up the Automated Traceability Graph and the Governance Dashboard. Neo4j is deployed; the Code Builder and CI Builder are implemented; the ATG is populated from the existing codebase and CI history. Grafana is deployed with the Constitutional Health Dashboard; Backstage is deployed with the per-subsystem Scorecards. The nightly self-audit is implemented and begins running. The OSCAL-AR feed is wired from the CI jobs (still in report-only mode from Phase 1) to the Grafana and Backstage data sources. The phase exits when the ATG is live with all 13 node types, the dashboards are live and showing real data, and the nightly self-audit is producing reports. Throughout Phase 2, the dashboards are observational; they do not yet gate anything.

8.6 Phase 3: Enforcement Cutover (Weeks 17-20)

Phase 3 is the cutover: Conftest switches from report-only to blocking; the Production Acceptance Gate becomes a GitHub Environment with Required Checks; the RC1 human review is retired for criteria that are now machine-enforced. The cutover is staged: first, the SEV-3 criteria are cut over (lowest risk); then SEV-2; then SEV-1. At each stage, the cutover is paused if the machine evaluation produces a false-positive block (a FAIL the human would not have raised). The phase exits when all GATE criteria are enforced by CI, the RCA's sign-off is a GitHub button click (not a hand-evaluation), and the RC1 human

review is formally retired for automated criteria. The RC1 documents remain as generated views (produced by the Document Generators from the MRC), but they are no longer the working surface of the engineering organization. The working surface is now the Dashboard, the ATG, and the CI gates.

8.7 Risk Mitigation

The migration carries risks that the phased approach is designed to mitigate:

- **Risk: False-positive blocks.** a Rego policy incorrectly encodes a provision, blocking legitimate work. Mitigation: Phase 1's report-only mode and parity measurement catch encoding errors before they block. Each policy has a documented escape hatch (a way to mark a criterion N/A with justification, reviewed by the RCA).
- **Risk: Graph inaccuracy.** the ATG is incomplete or incorrect, producing false integrity violations. Mitigation: Phase 2 runs the ATG in observation mode; integrity violations are reported but do not block until Phase 3.
- **Risk: Alert fatigue.** the nightly self-audit produces noise (too many findings, most false). Mitigation: the self-audit's findings are tuned during Phase 2; only findings above a confidence threshold are escalated.
- **Risk: CI dependency.** a CI outage means no criteria are evaluated, blocking all releases. Mitigation: the RC1 human review is retained as a fallback for SEV-1 criteria until Phase 3 is complete; a documented break-glass procedure permits the CSA to authorize a release during a CI outage with recorded justification.
- **Risk: Cultural resistance.** engineers resist the new regime, perceiving it as overhead or mistrust. Mitigation: the Backstage Scorecard gives developers immediate visibility into their service's compliance, turning the regime from a gate into a tool; the shift-left framing emphasizes that catching issues at PR time is cheaper than at release time.

Chapter 9 — Counterarguments and Limitations

9.1 Purpose

This chapter addresses the limitations of the RC2 evolution and the counterarguments against it. The RC2 architecture is not a panacea; it introduces new dependencies, new failure modes, and new costs. A rigorous roadmap acknowledges these honestly and specifies how they are mitigated, rather than presenting the evolution as unalloyed improvement. The limitations are not reasons to abandon RC2; they are reasons to implement it carefully, with the mitigations specified.

9.2 Limitation: The Machine-Readable Constitution Is Not the Constitution

The MRC encodes the provisions, but the provisions' meaning is not fully captured by the schema. A Rego policy that says “COUNT(dois) = 1” encodes the letter of INV-I-02 but not its spirit (that a DOI is a permanent citation handle for a scholarly work). An engineer who games the policy — minting a DOI that satisfies the count but is not registered with Crossref — has technically passed the criterion while violating the invariant's intent. The mitigation is two-fold: (1) the Rego policy includes the Crossref reconciliation check, not just the count, so gaming requires both a local count match and a Crossref match; (2) the human review is retained for judgement-based criteria that no machine can evaluate (e.g., “is this retraction notice substantively adequate?”). The MRC is the Constitution's executable form, not its complete meaning; the human-readable documents retain their role as the authoritative statement of intent.

9.3 Limitation: Automation Has Its Own Failure Modes

The RC2 architecture replaces human failure modes (inattention, sampling bias, fatigue) with machine failure modes (misconfiguration, false positives, cascading failures). A Rego policy with a bug blocks all PRs until fixed. A Neo4j outage means no integrity verification. A GitHub Actions outage means no CI gates. These are not hypothetical: CI outages and policy bugs are common in organizations that have adopted policy-as-code. The mitigation is the phased cutover (Phase 3 stages by severity), the break-glass procedure (CSA authorization with recorded justification), and the retention of the RC1 human regime as a fallback for SEV-1 criteria until the RC2 machinery has demonstrated reliability. The goal is not zero failure; the goal is fewer failures than the human regime, with faster detection and remediation.

9.4 Limitation: The Technology Stack Is a New Dependency

RC2 introduces dependencies on OPA, Neo4j, Backstage, Grafana, Sigstore, and OSCAL. Each is a mature, well-governed open-source project (OPA and Backstage are CNCF projects; Sigstore is OpenSSF; OSCAL is NIST), but each is a dependency nonetheless. A vulnerability in any of them, a licensing change, or a project deprecation would affect Opus Publica. The mitigation is the Constitution's technology-neutrality at the specification layer: the MRC's YAML/JSON Schema encoding is portable to any policy engine, any

graph database, any dashboard. If OPA is replaced by another engine, the Rego policies are rewritten, but the MRC is unchanged. If Neo4j is replaced by Neptune, the graph builders are adapted, but the graph model is unchanged. The technology stack is a layer below the MRC; the MRC is the durable asset, the stack is the implementation.

9.5 Limitation: Not Every Criterion Can Be Automated

Some criteria are judgement-based: “is the retraction notice substantively adequate?” “is the author's conflict-of-interest declaration complete?” “is the article's abstract an accurate summary of the content?” These are not machine-verifiable; they require human judgement. The RC2 architecture does not eliminate human review for these criteria; it eliminates human review for the criteria that ARE machine-verifiable, freeing human attention for the criteria that are not. The post-RC1 review's directive “eventually the computer should perform almost all of this” is the goal; “almost all” acknowledges that some criteria remain human. The MRC's verifier field distinguishes automated from human criteria, and the Dashboard surfaces the human-verified criteria as a distinct workload for the RCA.

9.6 Counterargument: “This Is Over-Engineering for a Publishing Platform”

A critic might argue that the RC2 architecture — OPA, Neo4j, Backstage, OSCAL, SLSA, Sigstore — is excessive for a scholarly publishing platform, appropriate for aerospace (DO-178C) or medical devices (FDA 21 CFR Part 820) but not for journals. The response is that scholarly publishing IS a regulated, audit-sensitive domain: the published record is the foundation of academic citation, scientific reproducibility, funder compliance, and tenure decisions. A platform that corrupts the record corrupts the citations, the research, and the careers. The regulated industries that adopted continuous assurance (aerospace, medical, financial) did so because the cost of corruption exceeded the cost of the machinery. Scholarly publishing's cost of corruption — a lost article, a duplicate DOI, a silently modified PDF — is borne by the scholarly community, not by the platform, which is precisely why the platform must internalize the discipline. The RC2 architecture is not over-engineering; it is the engineering that the domain's stakes demand.

9.7 Counterargument: “The ROI Is Unclear”

A critic might argue that the RC2 architecture's return on investment is unclear: the cost of building and operating it (engineer-time, infrastructure, training) is concrete and immediate, while the benefit (avoided corruption incidents, faster audits, funder compliance) is probabilistic and deferred. The response is twofold. First, the RC1 regime already incurs the cost: the 1,285-page hierarchy is maintained by human labor, the per-release certification consumes RCA hours, and the periodic audits consume external-auditor fees. RC2 does not introduce cost; it converts variable human cost into fixed machine cost, which scales better as the platform grows. Second, the benefit is not purely probabilistic: RC2's continuous assurance reduces the mean time to detection of a corruption incident from “quarterly audit” to “nightly self-audit or real-time alert,” and the cost of a corruption incident undetected for a quarter (citations built on a corrupt record, funder-compliance failure, reputation damage) is far higher

than the cost of the machinery that would have detected it in an hour. The ROI is positive; it is just denominated in avoided loss rather than in generated revenue.

9.8 Counterargument: “This Locks Us Into a Specific Stack”

A critic might argue that adopting OPA, Neo4j, Backstage, and the rest locks Opus Publica into a specific technology stack, contradicting the Constitution's technology-neutrality. The response, as noted in section 9.4, is that the technology-neutrality is preserved at the MRC layer: the MRC's YAML/JSON Schema encoding is portable. The technology stack is an implementation choice, documented in ADRs, and replaceable without amending the Constitution. The ADRs specify the migration path for each component (e.g., OPA → alternative engine requires rewriting Rego policies; Neo4j → Neptune requires adapting Cypher queries to openCypher). The stack is a layer, not a lock-in; the MRC is the durable asset.

Chapter 10 — Conclusion and Implications

10.1 Summary of Findings

This Roadmap has specified the RC2 evolution of Opus Publica's governance from human-enforced to self-enforcing. The evolution consists of five pillars — machine-readable constitution, automated traceability graph, continuous certification engine, governance dashboard, and self-auditing platform — supported by a technology stack of mature, well-governed open-source projects (OPA, Neo4j, Backstage, Grafana, Sigstore, OSCAL) and grounded in established standards (NIST SP 800-137 ISCM, NIST SP 800-204C policy-as-code, SLSA, JSON Schema 2020-12, OpenAPI 3.1, Conventional Commits, SemVer, RFC 6902). The migration is a 20-week phased transition that introduces no governance gap and mitigates the risks of false-positive blocks, graph inaccuracy, alert fatigue, CI dependency, and cultural resistance.

The RC2 architecture does not replace the RC1 Constitution; it operationalizes it. The 1,285-page hierarchy remains the authoritative statement of the platform's provisions. The MRC is the executable encoding of those provisions; the ATG is the live traceability graph; the CCE is the automated certification; the Dashboard is the real-time visibility; the SAP is the continuous assurance. Together, they convert the Constitution from a document humans read into a system machines execute, with humans reserved for judgement rather than verification. This is the constitutional inflection point the post-RC1 review identified, and it is the foundation on which RC2's product features (multi-tenancy, plugins, AI assistance) will be built and certified.

10.2 Implications for Engineering Practice

RC2 changes the daily practice of every engineer at Opus Publica. An engineer opening a pull request no longer wonders whether the PR satisfies the Constitution; the ConfTest gate tells them, in seconds, with a link to the specific provision. A tech lead no longer tracks compliance by reading the Certification Checklist; the Backstage Scorecard shows the service's grade in real time. The RCA no longer evaluates 600+ criteria by hand per release; the CCE evaluates them continuously, and the RCA intervenes only on exceptions. The on-call engineer no longer discovers an invariant violation during a quarterly audit; the SAP detects it in an hour and pages immediately. The external auditor no longer samples the traceability matrix; the auditor queries the ATG and receives an exhaustive, evidence-backed answer in milliseconds. The Constitution, in RC2, is not a document an engineer consults; it is a system that consults the engineer, telling them what is required, what is satisfied, and what is broken, continuously and without bias.

10.3 Implications for Audit and Compliance

RC2 positions Opus Publica as auditable by design in the strongest sense: not merely built to a standard, but continuously proving its compliance to the standard, with evidence that is fresh, exhaustive, and machine-checkable. The OSCAL-AR feed makes the certification legible to any auditor with OSCAL

tooling, including FedRAMP assessors, funder-compliance officers, and indexer-eligibility reviewers (DOAJ, Scopus, Web of Science). The SLSA L3 provenance and Sigstore signing make the supply chain verifiable end-to-end, satisfying the integrity requirements that preservation services (CLOCKSS, LOCKSS, Portico) and indexers increasingly demand. The continuous-assurance discipline (NIST SP 800-137 ISCM) makes Opus Publica's governance legible to the professional audit community (ISACA, IIA) in their own vocabulary. RC2 is not merely an engineering evolution; it is a compliance and auditability evolution that positions the platform for the scholarly-infrastructure ecosystem's highest trust tiers.

10.4 Future Outlook

RC2 is not the end of the evolution. The RC3 horizon, building on RC2's self-enforcing foundation, includes: (1) AI-assisted policy authoring — an LLM that drafts Rego policies from natural-language provision statements, with human review; (2) predictive compliance — ML models that forecast which criteria are at risk of failing before they fail, enabling preventive remediation; (3) cross-platform constitutional federation — the OSCAL-AR feed shared (with consent) across the scholarly-infrastructure ecosystem, so that a funder or indexer can verify a journal's compliance without a manual audit; (4) constitutional simulation — a tool that simulates the impact of a proposed constitutional amendment on the platform's compliance posture before the amendment is adopted. These are speculative; they depend on RC2's success. But RC2 is the foundation that makes them conceivable. Without a machine-readable constitution, there is nothing for an LLM to author; without continuous certification, there is no data for predictive models; without OSCAL, there is no format for federation. RC2 is the precondition for the future.

10.5 Call to Action

The Engineering Leadership Team SHALL:

12. **Action 1.** Approve this Roadmap as the RC2 governance architecture of record, subject to amendment through the Evolution Governance process (Constitution Vol II Ch 36).
13. **Action 2.** Commission the Architecture Decision Records for each technology choice (OPA, Neo4j, Backstage, Grafana, Sigstore, OSCAL), with the trade-offs and migration paths documented.
14. **Action 3.** Resource the Phase 0 team (2 engineers, 4 weeks) to stand up the MRC repository and encode the 85 backbone provisions.
15. **Action 4.** Establish the RC2 migration governance: a weekly migration review with the CSA, a parity dashboard (Phase 1), and a go/no-go decision at each phase boundary.
16. **Action 5.** Communicate the migration to the engineering organization, emphasizing the shift-left benefit (catching issues at PR time rather than release time) and the cultural change (from human verification to human judgement).

The RC1 Constitution established that Opus Publica is governed by a written, binding, comprehensive standard. RC2 establishes that the standard is enforced by the platform itself, continuously, exhaustively, and without the biases and gaps of human enforcement. This is the constitutional inflection point. The path forward is specified; the work begins.

References

The following references are the canonical authorities cited in this Roadmap. All URLs were verified during research (August 2026). Where a standard has been updated, the current canonical version is cited with the historical version noted.

Standards and Specifications

NIST SP 800-137. Information Security Continuous Monitoring (ISCM) for Federal Information Systems and Organizations. <https://csrc.nist.gov/pubs/sp/800/137/final>

NIST SP 800-204C. Implementation of DevSecOps for a Microservices-based Application with Service Mesh (defines “policy as code”). <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-204c.pdf>

NIST SP 800-92. Guide to Computer Security Log Management. <https://csrc.nist.gov/pubs/sp/800/92/final>

NIST OSCAL. Open Security Controls Assessment Language. <https://pages.nist.gov/OSCAL/>

OWASP ASVS 4.0. Application Security Verification Standard. <https://owasp.org/www-project-application-security-verification-standard/>

JSON Schema Draft 2020-12. JSON Schema specification. <https://json-schema.org/draft/2020-12>

YAML 1.2.2. YAML Ain't Markup Language specification. <https://yaml.org/spec/1.2.2/>

OpenAPI 3.1. OpenAPI Specification (aligned with JSON Schema 2020-12). <https://swagger.io/specification/>

RFC 2119. Key words for use in RFCs to indicate requirement levels. <https://datatracker.ietf.org/doc/html/rfc2119>

RFC 6902. JSON Patch. <https://www.rfc-editor.org/info/rfc6902>

RFC 7386. JSON Merge Patch. <https://datatracker.ietf.org/doc/html/rfc7386>

RFC 9110. HTTP Semantics. <https://datatracker.ietf.org/doc/html/rfc9110>

RFC 8446. TLS 1.3. <https://datatracker.ietf.org/doc/html/rfc8446>

ISO/IEC 5962:2021. SPDX (internationally recognized SBOM standard). <https://www.linuxfoundation.org/press/featured/spdx-becomes-internationally-recognized-standard-for-software-bill-of-materials>

ECMA-424. CycloneDX (OWASP-originated BOM standard, now ECMA-standardized). <https://owasp.org/www-project-cyclonedx/>

ANSI/NISO Z39.96-2021. JATS (Journal Article Tag Suite). <https://www.niso.org/standards-committees/jats>

DO-178C. Software Considerations in Airborne Systems and Equipment Certification (RTCA). <https://www.rtca.org/>

FDA 21 CFR Part 820. Quality Management System Regulation. <https://www.ecfr.gov/current/title-21/chapter-I/subchapter-H/part-820>

Conventional Commits 1.0.0. A specification for adding human and machine meaning to commit messages. <https://www.conventionalcommits.org/en/v1.0.0>

Semantic Versioning 2.0.0. A formal specification for version numbers. <https://semver.org/spec/v2.0.0.html>

OASIS Naming Guidelines. Metadata and Versioning for OASIS specifications. <https://docs.oasis-open.org/specGuidelines/namingGuidelines/metadata.html>

Technologies and Projects

Open Policy Agent (OPA). CNCF Graduated policy engine; Rego policy language. <https://www.openpolicyagent.org/>

Conftest. OPA subproject for testing structured configuration data. <https://www.conftest.dev/>

CUE. Configure, Unify, Execute — Google's data-validation language. <https://cuelang.org/>

Backstage. Spotify open-source developer portal framework (CNCF). <https://backstage.io/>

Neo4j. Market-leading property-graph database; Cypher query language. <https://neo4j.com/>

OpenTelemetry. CNCF vendor-neutral observability framework. <https://opentelemetry.io/>

Prometheus. CNCF Graduated monitoring and alerting toolkit. <https://prometheus.io/>

Grafana. Open-source visualization and dashboarding platform. <https://grafana.com/>

Flux. CNCF Graduated GitOps controller. <https://fluxcd.io/>

Argo CD. CNCF Graduated GitOps controller. <https://argoproj.github.io/argo-cd/>

OpenGitOps. CNCF GitOps Working Group principles. <https://opengitops.dev/>

SLSA. Supply-chain Levels for Software Artifacts (OpenSSF). <https://slsa.dev/>

Sigstore. OpenSSF code-signing stack (Cosign, Rekor, Fulcio). <https://www.sigstore.dev/>

in-toto. Software supply-chain integrity framework (OpenSSF). <https://in-toto.io/>

CycloneDX. OWASP-originated SBOM standard (ECMA-424). <https://cyclonedx.org/>

SPDX. Linux Foundation SBOM standard (ISO/IEC 5962:2021). <https://spdx.dev/>

driftctl. Infrastructure drift detection (Snyk). <https://github.com/snyk/driftctl>

yq. Go-based YAML/JSON processor (mikefarah/yq). <https://github.com/mikefarah/yq>

jq. Command-line JSON processor. <https://jqlang.github.io/jq/>

GitHub Actions. GitHub CI/CD platform. <https://docs.github.com/en/actions>

Scholarly Publishing Infrastructure

Crossref. DOI registration agency; REST API and deposit API. <https://www.crossref.org/>

Crossref REST API. Metadata retrieval API (replaces deprecated Event Data). <https://www.crossref.org/documentation/retrieve-metadata/rest-api/>

ORCID Public API. Free read-only access to public ORCID records. <https://info.orcid.org/what-is-orcid/services/public-api>

ORCID Member API. Read/write access for ORCID member organizations. <https://info.orcid.org/what-is-orcid/services/member-api>

CLOCKSS. Controlled LOCKSS dark archive. <https://clockss.org/>

LOCKSS. Lots Of Copies Keep Stuff Safe — decentralized preservation. <https://www.lockss.org/>

DOAJ. Directory of Open Access Journals. <https://doaj.org/>

OJS (Open Journal Systems). PKP open-source journal management software. <https://pkp.sfu.ca/ojs/>

COPE. Committee on Publication Ethics. <https://publicationethics.org/>

Industry and Conceptual References

Google SRE Book. Site Reliability Engineering (free online). <https://sre.google/sre-book/>

Google SRE Workbook. Implementing SLOs; alerting on SLOs. <https://sre.google/workbook/>

ISACA. Continuous assurance in cloud compliance. <https://www.isaca.org/resources/news-and-trends/isaca-now-blog/2025/cloud-compliance-continuous-assurance-harnessing-ai-rpa-and-cspm-for-a-new-era-of-trust>

Confluent Schema Registry. Schema evolution and compatibility types. <https://docs.confluent.io/platform/current/schema-registry/fundamentals/schema-evolution.html>

Microsoft Azure EPAC. Enterprise Policy as Code (production reference). <https://azure.github.io/enterprise-azure-policy-as-code>

— Issued under the authority of the Chief Systems Architect, Opus Publica — RC2 Evolution Roadmap —